ChatGPT

Incorporating C64 VIC-II and SID into Modern Hardware

Bringing the Commodore 64's VIC-II video chip and SID sound chip into a modern project can recreate the authentic 8-bit audiovisual aesthetic. The VIC-II (Video Interface Chip II) generated the C64's graphics, including sprites, scrolling backgrounds, and raster effects, while the SID (Sound Interface Device) produced its distinctive chiptune music and sound effects [1] [2]. Below we outline **hardware-level approaches** to interface or replicate these chips (using real chips, microcontrollers, or FPGAs) and **software-level techniques** (6502 assembly or modern code) to program classic visual effects (sprites, raster bars, smooth scrolling) and audio effects (chiptune synthesis with waveforms, filters, and envelopes).

Hardware-Level Approaches to VIC-II and SID

Using Original Chips: One approach is to obtain actual MOS 6567/6569 VIC-II and 6581/8580 SID chips and build them into a new hardware design. This requires recreating the necessary supporting environment (clock signals, bus logic, memory, etc.). The VIC-II is a complex bus-mastering video processor that shares the system bus with the CPU – in a C64 it alternates memory access with the 6510 CPU on each half-clock cycle and can halt the CPU when it needs extra cycles [3]. To use a VIC-II on its own, you essentially need to give it a 1 MHz 6502-like bus to control (including RAM for screen data and sprite patterns). Hobbyists have built minimal C64 motherboards or "Frankenstein" setups using a 6502/6510 CPU, level translators, and static RAM to get a VIC-II running outside of an original C64. Keep in mind the VIC-II also needs a precise clock source (≈17.7 MHz PAL or 14.3 MHz NTSC, divided down for the CPU) and generates video signals (composite or Y/C) that must be output to a display.

The SID chip is somewhat easier to interface since it's essentially an audio synthesizer that can be treated as an I/O-mapped peripheral. The SID requires a **~1 MHz clock** signal and a parallel 8-bit data bus for writing its 29 registers, plus stable 5V (and 9V or 12V depending on chip type) power. For example, an Arduino-based project interfacing a MOS 6581 SID used an **external 1 MHz crystal oscillator** (or a timer pin toggling at 1 MHz) to clock the SID [4]. The Arduino can then write to the SID's registers by bit-banging or using helper hardware: one open-source Arduino SID shield uses dual 8-bit shift registers to load address and data lines, with a latch to update the SID's bus lines while the 1 MHz clock is high [5] [6]. This allows sending values to the SID's 24 write-able registers (such as setting oscillator frequency or envelope parameters) under program control [7]. In general, when using a real SID chip, **writes are timed with the clock** – the chip latches data on the appropriate phase of the 1 MHz cycle [6]. Several DIY projects have successfully driven SID chips with microcontrollers: for instance, the **MIDIbox SID** synthesizer uses PIC microcontrollers to control one or multiple SID chips as a MIDI-controlled sound module [6]. Arduino enthusiasts have also created SID player shields and libraries to play music through a SID (some even emulate the SID in software on faster MCUs when a real chip isn't available) [8].

Modern drop-in replacements are available if original chips are hard to find. For the SID, there are **microcontroller or FPGA-based clones** like **SwinSID**, **ARMSID**, and **SIDKick** that emulate the SID's functionality and plug into the C64's SID socket [9]. These often run on modern hardware (AVR, ARM, or

FPGA) and output analog audio, simplifying integration (some run on a single 5V supply, for example). For the VIC-II, reproductions are more complex due to video output, but projects exist – notably the **VIC-II Kawari** which is an FPGA-based VIC-II replacement board. It plugs into the VIC-II socket on a C64 motherboard and implements all the VIC-II's functions (bus arbitration, DRAM refresh, light pen input, clock generation, etc.) in an FPGA [10] [11]. This project provides modern video outputs (VGA or HDMI) while remaining cycle-accurate to support all the original graphics tricks [12] [13]. The open-source **VIC-II Kawari core** (in Verilog) is available with PCB designs ranging from simple to feature-rich [14]. In a modern build, one could use such an FPGA core *standalone*, feeding it with data from a microcontroller or a soft-CPU. Similarly, the **FPGA SID (FPGASID)** project implements two SID voices in VHDL (for stereo) with very high fidelity, which can be integrated into an FPGA-based system [15].

Connecting via Microcontrollers or Co-Processors: If using real chips directly, one strategy is to dedicate a microcontroller (or a 65C02 co-processor) to act as the "CPU" for the VIC-II and SID. For example, a 65C02 running at 1 MHz (or an emulated 6502 core on a microcontroller/FPGA) can be used to run 6502 assembly code that manipulates VIC-II/SID registers in real time. Alternatively, a fast modern MCU could bit-bang the bus signals; however, ensuring precise timing at 1 MHz and servicing the VIC-II's constant memory fetches is **non-trivial** for a typical MCU [16] [17]. One recommendation is to use MCUs with an external memory interface (e.g. some Atmel/Microchip AVR Mega or SAM parts) which can mimic the 6502 bus. Even so, building a stable dual-master bus (the VIC-II and CPU sharing RAM) may require glue logic or CPLDs – it's often easier to use an FPGA to handle bus arbitration if you're set on a custom hardware solution. In summary, **driving a SID chip from a microcontroller is quite feasible** (as it only needs writes at manageable speeds), but **driving a VIC-II** often means **essentially recreating a C64's digital logic** or using an FPGA core, due to the VIC-II's tight integration with memory and timing.

Software-Level Approaches and Programming Techniques

6502 Assembly Programming: On the software side, achieving authentic C64 effects typically involves programming in 6502/6510 assembly language. The VIC-II and SID are controlled by writing values to their memory-mapped registers in the C64's address space. The VIC-II has 47 control registers mapped at addresses $D000–$D02E (54272–54318 decimal) [18], of which 34 are devoted to sprite control [18]. The SID has 29 registers at $D400–$D41C (54272–54300 decimal), including frequency, waveform, envelope, and filter controls for its three voices [19] [20]. In the C64 era, one would use `POKE` in BASIC or, more effectively, assembly `LDA/STA` instructions to set these registers. For example, POKEing a 16-bit value into $D400/$D401 sets voice 1's frequency, and setting the high nybble of $D404 adjusts that voice's attack rate [21]. Complex visual/sound effects required **cycle-accurate** assembly code and clever use of interrupts. Programmers often set up a raster interrupt (using the VIC-II's interrupt register at $D019/$D01A) to trigger an ISR at a specific scanline, so that register changes occur at exact screen positions [22]. Mastery of such raster timing is *essential* for advanced VIC-II tricks [22]. Many classic demo effects (sprite multiplexing, border effects, color splits) are done by "racing the beam," i.e. synchronizing code to the raster beam's position.

Today, you can write and compile 6502 assembly on a PC using cross-assemblers (such as ACME or [KickAssembler]) and test in an emulator (like VICE) before running it on your custom hardware. There are also C cross-compilers (like *cc65*) and even modern higher-level languages (e.g. *KickC*, a C-like language that targets 6502) that can simplify parts of the coding, though the most timing-critical parts usually require hand-tuned assembly. Another modern approach is to embed a **soft-core 6502** in an FPGA along with the VIC-II/SID cores so that you can run original C64 code or new assembly on your custom device. For instance,

the open-source **C64 FPGA implementations** (such as the MISTer's C64 core or fpga64) include a cycle-accurate 6510 CPU, VIC-II, and SID – essentially a "C64 on a chip." You could use such a core and then interface it with external systems (for example, some projects expose a fast serial or SPI link to a PC or host MCU for feeding data or memory content). This way, you can leverage the huge knowledge base of C64 demo programming to produce effects, while the FPGA handles the low-level timing.

High-Level Control and APIs: If you prefer not to write raw assembly, you can still manipulate the chips at a low level using a higher-level language on a microcontroller or PC, by sending the right values to the hardware registers. For example, the Arduino SID library mentioned earlier provides a friendly API to set SID parameters (waveform, frequency, ADSR, etc.) in C++ code, abstracting the register writes [7]. On the video side, there aren't comparably simple libraries for VIC-II (due to its complexity), but one could conceive a driver where a microcontroller sends commands to an FPGA-based VIC-II (for instance, writing to a register file that the VIC-II core reads from). Another "software" route is to emulate the chips entirely in software (using sound and graphics libraries on a microcontroller or single-board computer). For instance, one could use the *ReSID* software engine (used in VICE) to generate SID audio on a fast MCU, and use a small GPU or TFT display to mimic VIC-II-style graphics. However, this veers away from "using VIC-II and SID chips" directly – it's more of a reimplementation of their *behavior*.

In summary, **programming the VIC-II and SID** means carefully orchestrating writes to their registers to produce the desired effect. Whether you do that with true 6502 machine code or via a modern microcontroller sending data, the logic of **what to write and when** remains the same as on the C64. Next, we'll look at some classic effects and how they are achieved through these registers.

Classic Visual Effects with the VIC-II

The VIC-II was a very advanced video chip for its time, supporting a variety of graphics modes and features: - **Sprites:** The VIC-II can display **8 hardware sprites** (called "MOBs" in Commodore docs) on screen, each 24×21 pixels (or 12×21 in multicolor mode) [23]. Sprites have their own position registers ($D000–$D00F for X/Y coords) and attribute registers (color, size, etc.) [24] [25]. By setting these registers, you can make hardware-controlled objects (player characters, bullets, etc.) move independently of the background. The chip can handle all 8 sprites *per scanline*; if more overlap on the same line, an internal flag is set to indicate sprite collisions. A classic trick in demos is **sprite multiplexing**, which involves reusing sprites in different vertical zones of the screen. By waiting for the raster to reach a certain Y position (via an interrupt or cycle timing) and then quickly changing a sprite's Y register and pointer to new data, programmers achieved more than 8 visible sprites at once [22]. This requires precise timing but was used to create spectacular displays of dozens of sprites in mid-'80s demos. Similarly, the VIC-II allows sprite-sprite and sprite-background collision detection bits, which game programmers used for simple physics and triggers.

- **Scrolling and Screen Splits:** The VIC-II supports **smooth scrolling** in hardware – the registers $D011 and $D016 contain 3-bit vertical and horizontal fine-scroll values, which offset the display by 0–7 pixels [26]. Using these, one can scroll a tiled background seamlessly (when the scroll registers overflow, you'd swap in the next row or column of tiles in memory to create a continuous scroll). Many C64 games used pixel-level smooth scrolling for side-scrolling or vertical-scrolling backgrounds. The VIC-II also has a "row select" and "column select" bit (in $D011 and $D016) to toggle between 24 or 25 row text display, and 38 or 40 column display – by dynamically flipping these at runtime you can create *split-screen effects*. For example, a game might use a 3-line high score panel at the bottom with no scrolling, and a scrolling playfield above; this is done by changing

$D011/$D016 at a specific raster line to freeze scrolling for the bottom area. Indeed, by *reloading VIC-II registers on the fly each frame*, you can give different parts of the screen independent settings (separate scroll speeds, different color palettes, even different video modes in some cases) [22]. This technique is heavily used in demos to, say, have a static text panel and a flying graphics panel on the same screen.

- **Raster Bars: Raster bars** are a signature demo effect using the VIC-II (originally popular on Atari systems and then the C64). These are horizontal bars of color that animate up and down the screen. The C64 has one background color register ($D021) that applies to the whole screen background, and a border color register ($D020) for the screen border. By rapidly changing these color registers on specific scan lines, one can draw horizontal colored stripes. Essentially the CPU must "race the beam" – i.e. execute a store to $D021 at the exact moment the electron beam is drawing a particular line [27]. If you set $D021 to red just before line 100 is drawn, and then to blue before line 101, you'll get a red stripe on line 100 and blue on 101. Doing this with carefully chosen color sequences produces the classic multicolored bar effect across many lines. Often, a raster interrupt is set at the top of the area where bars should start, and the ISR will update $D021 (and perhaps $D020 for the border) every few lines to animate the bars. On a C64, the PAL machine gives 63 CPU cycles per line (NTSC gives 65) for the CPU to do any work during that line's drawing [28]. However, on every 8th line the VIC-II triggers a **"badline"** where it steals 40 CPU cycles to fetch a row of character or bitmap data [29]. This means the CPU has much less time on those lines to make color changes, complicating timing. Demo coders use cycle-exact delays or unrolled NOP loops to stabilize timing across badlines (or they choose to do color changes on non-badlines only). The result, when done right, is a set of flicker-free flat colored bars that can be moved or blended. Raster bars can also extend into the normally blank border area by using tricks to open the border (e.g. tricking the VIC-II into thinking sprites are active at the edges). This yields the full-screen striped background seen in many intro demos.

- **Graphical Modes and Tricks:** The VIC-II can operate in several modes: text mode (40×25 characters) with either single-color or multicolor characters, and bitmap mode (320×200 or 160×200 in multicolor) [30]. Programmers often leveraged character redefinition to draw graphics (by redefining the 8×8 pixel font tiles). A famous VIC-II trick for enhancing graphics is **FLI (Flexible Line Interpretation)** which allows more colors on a bitmap by using raster interrupts every 8 lines to switch video matrix pointers – this is advanced, but worth noting as a demo-oriented technique to push VIC-II beyond its design limits. Simpler effects include **color cycling** (changing the palette registers $D021–$D024 or $D025/$D026 sprite multi-colors over time to animate, say, a copper bar or a water effect) and **sprite multiplexing** as mentioned. The VIC-II's abilities to trigger interrupts on a given scanline [31] and to scroll and reuse sprites were the foundation of many trick effects. In modern terms, these are achieved by low-level register manipulation – which you can do either with actual 6502 code or by programming your microcontroller/FPGA to poke the values at the right times.

Classic Audio Effects with the SID

Block diagram of the MOS 6581/8580 SID sound chip architecture, showing its three independent oscillator/envelope voices and shared filter/output stage. Each voice has a waveform generator (pulse, sawtooth, triangle, or noise) with frequency and duty cycle registers, an ADSR envelope, and optional ring-modulation or sync with

another voice. All voices mix into a common analog filter (low-pass, band-pass, high-pass) and master volume output [32].

The SID chip is essentially a 3-voice analog synthesizer on a chip. It was designed by Bob Yannes with a musician's perspective, which is why it was far more advanced than other home computer sound chips of the early 1980s [33] [34]. Key features of the SID include: **three voices (oscillators)** each with 16-bit frequency control, **four waveform types** per voice (selectable sawtooth, triangle, pulse [with variable PWM], or pseudo-random noise) [32] [35], and a programmable ADSR **envelope generator** for each voice (Attack/Decay/Sustain/Release controlled by 4-bit values) [36]. Voices can also be combined through **ring modulation** and **hard synchronization**: each SID voice has bits to ring-modulate or sync it with another voice (voice 1->2, 2->3, 3->1 in a cyclical pattern) [37] [38]. Ring modulation in the SID multiplies one oscillator's output with another's, producing rich metallic sounds, while sync resets one oscillator's phase when another overflows, producing harsher, sync-buzz effects [39]. These features enabled complex tone generation like in no other 8-bit sound chip.

To program the SID, one writes to its register set at $D400–$D418. For example, to play a note, you would load the frequency registers for one voice ($D400 low byte, $D401 high byte for voice 1), select a waveform in the voice's control register ($D404 for voice 1 – bits choose triangle/saw/pulse/noise and set ring-mod or sync, plus the Gate bit) [40] [41], set the ADSR by writing the Attack/Decay ($D405) and Sustain/Release ($D406) registers, then finally set the **Gate bit** in the control register to 1 to trigger the note. The SID's envelopes are hardware ADSR – once you set Gate=1, the volume of that voice ramps up according to the Attack time to the peak, then decays to the Sustain level, and holds as long as Gate is 1. Clearing the Gate (0) initiates the Release phase to fade out the note [42] [43]. Because these envelope generators run independently once triggered, the 6510 CPU doesn't need to constantly shape the volume – it only needs to start/stop notes and perhaps update parameters for effects.

Chiptune Techniques: C64 musicians (and demo coders) developed a slew of techniques to get the most out of the SID's three voices. One common trick is **arpeggios**, rapid cycling of a voice's frequency to create the illusion of a chord. Because the SID has only 3 voices, playing a full triad chord would normally use all three; instead, composers would program a single voice to quickly alternate between three pitches (for example C, E, G frequencies in succession) at a rate of 50+ times a second. The result is a "buzzy" chord that became a hallmark of chiptune music. Another technique is using the programmable **pulse width** of the square wave to create rich, evolving timbres – modulating the pulse width over time (e.g. within an instrument's frame or via software LFO) produces a chorus-like or phaser effect. The SID's **analog filter** is another major element: it's a 12 dB/octave resonant multimode filter that all voices can be routed into [44]. You can choose low-pass, band-pass, and/or high-pass modes (even combine them for notch effects) and set a cutoff frequency (11-bit value across $D415–$D416) and resonance level (4-bit at $D417) [45]. By routing, say, two voices through the filter and leaving one voice unfiltered, musicians could have a bass line or effects with filtering while a lead voice remained bright. Automated filter sweeps (changing the cutoff in software during playback) yield the famous wah-wah or underwater sounds in C64 tunes. The combination of **ring-mod, sync, filter sweeps, and fast arpeggios** gives SID music its unique character [34] – these are all achievable by periodically updating the SID registers in code.

Percussion and PCM: Despite being primarily a tone generator, the SID was even pushed to play sampled audio. Clever hackers discovered you could exploit the SID's quirk (on 6581 chips) where changing the volume register ($D418) caused a slight DC offset pop. By rapidly changing the volume register, you can output 4-bit PCM sample data (each pop being a 4-bit amplitude) – effectively a **4-bit DAC** abuse [46]. This

was used to play back drums or speech samples in games (e.g. the voice in *Ghostbusters* or drum samples in *Arkanoid*). It consumes a lot of CPU to stream samples (because the 1 MHz CPU has to output bytes at ~8 kHz or higher), but it's a noteworthy technique. Modern SID musicians more often use the improved method of sample playback unveiled in 2008 which uses a waveform "DC" trick via the test bit for higher fidelity 8-bit samples [47], but that's beyond the scope here. For a retro project, you might not need sample playback – but if you do, it's possible with SID at the cost of heavy CPU use.

Modern control of SID: In a contemporary hardware project, you could drive the SID with MIDI or other inputs to recreate a synth. For example, the MIDIbox SID mentioned earlier allows using MIDI keyboard or sequencer data to play the SID – the microcontroller translates note on/off to SID register writes, implements things like LFOs or arpeggio tables in software, and uses the SID's analog sound generation for output. You could similarly connect a SID (or SID substitute like SwinSID) to a Raspberry Pi or Arduino that listens on USB or Bluetooth for events and then updates the SID registers accordingly to generate sound. There are also software tools on PC (GoatTracker, DefleMask, etc.) to compose SID music; these can output data or even drive hardware SID devices like the **SIDBlaster** USB interface (an open-source USB device that hosts a real SID chip for playback) [48]. Such tools and interfaces might be useful if your project aims to play existing SID tunes on demand, offloading the need to program every note by hand.

Resources and References

Building a retro-inspired project with VIC-II and SID is greatly aided by the wealth of documentation and community knowledge from decades of C64 development. Here are some valuable resources:

- **Official Documentation:** The *Commodore 64 Programmer's Reference Guide* and *Mapping the Commodore 64* are must-haves. They detail memory maps and have chapters on the VIC-II and SID registers and usage. For instance, the PRG includes example code for sprite setup, and SID register tables. Many of these are available as PDFs or online scans [49].

- **VIC-II Details:** The **MOS 6567/6569 VIC-II article by Christian Bauer** (available via archive at zimmers.net) is an excellent technical summary [50]. It explains how the VIC-II generates video, timing of badlines, etc. Also, the community-run C64 Wiki has a VIC article and also a "C64 Graphics" programming section with demo-coding techniques. For timing-critical effects, check out the *Codebase64* articles on raster timing and IRQ stabilizers.

- **SID Details:** The **SID chip datasheet** and the C64 Wiki's SID page provide comprehensive info on SID registers, waveforms, and differences between 6581 and 8580 chips [32] [20]. A classic read is the "**SID Primer**" series in Compute!'s Gazette (e.g. *Working with SID* by Craig Chamberlain) which explains SID synthesis with examples [51] [42]. For composers, tools like **SID Factory II** and **GoatTracker** allow creating music that you can use in your project; they output data in SID register change format or even ready-to-run 6502 code.

- **Community Projects:** Explore open-source projects like **randyrossi/vicii-kawari** on GitHub (open FPGA core for VIC-II) [14] and **SIDBlaster** or **SwinkiSID** repositories for SID. The **MIDIbox SID** project (at ucapps.de) is well-documented and includes schematics and code for a multi-SID synthesizer, which can be insightful if you plan on integrating MIDI or external control. There are also forums like Lemon64 and the 8-Bit Collective where people discuss interfacing C64 chips with modern tech.

- **FPGA/CPLD Cores:** If you plan to use an FPGA, you can leverage existing cores: the **Turbo Chameleon 64** and **Ultimate 64** (by Gideon) are examples of FPGA implementations of a C64 – not open-source, but they demonstrate that highly accurate emulation is possible in hardware. On the open side, MiSTer's FPGA C64 core or the older **FPGA64** project by Peter Wendrich [52] can be references (some are VHDL, some Verilog). These cores include VIC-II, SID, CIAs, etc., but you can choose to instantiate just the pieces you need (say, VIC-II and a simplified CPU to drive it).

- **Safety and Level Shifting:** One practical note – if wiring original chips, remember the VIC-II and SID are NMOS chips that run at 5V and are static-sensitive. The 6581 SID needs 12V and 5V, while the 8580 needs 9V and 5V [53], and audio output from SID is an analog voltage that typically should go through an amplifier or at least a filtering capacitor to remove DC bias. Ensure you provide the correct supply voltages and use level shifters if your controlling device is 3.3V logic. Also, the VIC-II generates video at 15kHz horizontal frequency – to use a modern display, you'll need a upscaler or an FPGA that can convert it to VGA/HDMI (as Kawari does). Otherwise, plan for an old-style CRT or a composite input monitor to view the raw VIC-II output.

By combining these **hardware techniques** (real chips or modern reimplementations) with **software know-how** (6502 assembly or equivalent control logic), you can authentically reproduce effects like a multi-color sprite bouncing on a pixelated background with chip tune music playing – essentially bringing the soul of the Commodore 64 into your modern project. The demoscene and retro computing community has accumulated vast knowledge, so don't hesitate to use existing open-source code and ask in forums; you'll find people who have interfaced SIDs to Raspberry Pis, or put VIC-II cores in Arduino-sized FPGAs. With patience and careful study of the VIC-II's timing and the SID's registers, you'll be able to implement everything from **smooth scrolling tilemaps** to **moving raster bars** with accompanying **SID melodies**, giving your project an authentic 1980s flavor. Good luck, and happy hacking!

Sources: The information above is drawn from C64 official documentation, community wiki articles, and modern projects. Key references include the Commodore 64 Programmer's Reference Guide, Christian Bauer's VIC-II notes [50], C64 Wiki's VIC-II and SID pages [18] [32], the VIC-II Kawari project docs [11], and the Arduino SID interface library by Charlotte Gore [5], among others as cited in-line. These provide in-depth technical details on registers and usage for anyone looking to dive deeper. Enjoy bringing these classic chips to life in the modern era!

---

[1] [3] [18] [22] [23] MOS Technology VIC-II - Wikipedia
https://en.wikipedia.org/wiki/MOS_Technology_VIC-II

[2] [34] [35] [37] [38] [39] [44] [46] [47] MOS Technology 6581 - Wikipedia
https://en.wikipedia.org/wiki/MOS_Technology_6581

[4] [5] [7] GitHub - CharlotteGore/MOS6581: C++ Arduino library for controlling a MOS 6581 etc via SPI interface
https://github.com/CharlotteGore/MOS6581

[6] [16] [17] Interface to a MOS 6581 (SID chip from a C64) so I can use it as synth - General Guidance - Arduino Forum
https://forum.arduino.cc/t/interface-to-a-mos-6581-sid-chip-from-a-c64-so-i-can-use-it-as-synth/650297

8 Arduino SID Player: Play C64 Music on Your Uno - Maker Hacks
https://www.makerhacks.com/arduino-sid-player/

9 19 20 32 33 40 41 45 53 SID - C64-Wiki
https://www.c64-wiki.com/wiki/SID

10 12 13 VIC-II Kawari
http://accentual.com/vicii-kawari/

11 14 GitHub - randyrossi/vicii-kawari: Commodore 64 VIC-II 6567/6569 Replacement Project
https://github.com/randyrossi/vicii-kawari

15 FPGASID - The better SID, KryoFlux Web Store
https://webstore.kryoflux.com/catalog/product_info.php?products_id=63

21 36 42 43 51 Working with SID
https://www.atarimagazines.com/compute/issue41/Working_With_SID.php

24 25 26 30 50 VIC-II reference
https://www.oxyron.de/html/registers_vic2.html

27 28 29 Galactic Rasterbar Power | Obliterator918's Commodore 64 Project Haven
https://www.obliterator918.com/galactic-rasterbar-power/

31 Raster bar - Wikipedia
https://en.wikipedia.org/wiki/Raster_bar

48 SIDBlaster-USB Nano - CBMretro
https://cbmretro.fi/product/sidblaster-usb-nano/

49 [PDF] Commodore 64 Programmers Reference Manual
https://www.commodore.ca/manuals/c64_programmers_reference/c64-programmers_reference_guide-04-programming_sound.pdf

52 Syntiac pages - FPGA-64
https://www.syntiac.com/fpga64.html